UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2016-12, page [7]

Unofficial model answer:

a) Advantage of SMP applied in NUMA: In NUMA, as in SMP, all processors share a single global physical address space and can access all main memory, so the same program/OS model applies and any processor can run any task and reach any data (a single system image). This gives SMP's key benefit — easy load sharing and uniform programming model — while the distributed memory banks reduce contention on one central memory bus.

b) Disadvantage of cluster architecture: A cluster is a collection of independent computers connected by a network, so inter-node communication has much higher latency and lower bandwidth than shared-memory access, and there is no single shared physical address space. Consequently message passing/software must be written for distributed memory, and the system is harder to program and manage (plus network is a potential bottleneck/failure point).

c) Possible cluster architecture based on Figure 3: Take the NUMA figure's node (processors + local main memory) and replicate it as separate complete computers, then connect the computers through a high-speed interconnection network (e.g., LAN/switch) instead of the NUMA interconnect. Each node has its own CPU(s) and private main memory and its own OS image; nodes communicate by messages over the network. (Diagram: [CPU+Memory]--[CPU+Memory]--... joined by a network/switch.)

Marking points:

• 1 mark (a): states that all processors share a single global address space / any processor can access any memory (SMP-style uniform programming and load sharing).

• 1 mark (b): identifies high inter-node communication latency / no shared memory (distributed memory, message passing) as the disadvantage.

• 1 mark (b): explains consequence — harder programming/management or network bottleneck.

• 1 mark (c): draws multiple independent nodes, each with its own processor(s) and local main memory.

• 1 mark (c): shows nodes connected by a network/interconnection (switch/LAN), not a shared bus as in Figure 3.

Model confidence: medium.

Model note: Figure 3 itself was supplied only as a caption/crop reference, so part (c) is answered generically by replicating the NUMA node as independent computers joined by a network. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 2 (5 marks)

Source: CSC429-2019-12, page [2]

Unofficial model answer:

a) Inventor 'Y' is Blaise Pascal. The clue (French mathematician/physicist, 1642, built a calculating machine to help his father with tax accounting) identifies Pascal and his Pascaline.

b) Another calculating machine from the same generation (the mechanical/gears-and-wheels era of the 17th century) is the Leibniz wheel / Stepped Reckoner, invented by Gottfried Wilhelm Leibniz (c. 1672). (The Pascaline itself is Pascal's machine; the Leibniz Stepped Reckoner is the standard companion answer for this generation.)

c) The programming language named after Pascal is Pascal (developed by Niklaus Wirth, named in honour of Blaise Pascal).

d) Two mathematical operations the Pascaline could perform are addition and subtraction. (It carried

© Hak Cipta Universiti Teknologi MARA

automatically between digits; multiplication and division were done by repeated addition/subtraction rather than directly.)

Note: The figure is supplied only by caption/extracted text, so identification relies on the textual clues rather than the image itself.

Marking points:

• 1 mark (a): Correctly identifies inventor 'Y' as Blaise Pascal.

• 1 mark (b): Names a valid same-generation calculating machine, e.g. Leibniz Stepped Reckoner / Leibniz wheel (accept other correct 17th-century mechanical calculator).

• 1 mark (c): States the Pascal programming language.

• 2 marks (d): One mark each for two correct operations — addition and subtraction (accept multiplication/division only if justified as repeated add/subtract).

Model confidence: high.

Model note: Figure 1 was available only as caption/extracted text, not as an inspectable image; answer is based on the textual clues. Part (d) marks split 1+1 for the two operations. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 3 (5 marks)

Source: CSC429-2025-07, page [3]

Unofficial model answer:

The operands are -9B (hex) and -37a (hex). First convert each to binary using 8 bits (since both magnitudes fit in 8 bits): $9B_{16}$ = 1001 1011_2, so -9B in 8-bit 1's complement = 0110 0100_2, and in 8-bit 2's complement = 0110 0101_2. $37a_{16}$ = 0011 0111 1010_2, which needs 10 bits; using 10-bit representation, -37a in 1's complement = 1100 1000 0101_2, and in 2's complement = 1100 1000 0110_2. To use a common width, sign-extend -9B to 10 bits: 1's complement 11 0110 0100_2, 2's complement 11 0110 0101_2. The required operation is (-9B) - (37a) = (-9B) + (-37a). 1's complement workflow: add the two 10-bit 1's complement values: 11 0110 0100 + 11 1000 0101 = 1 10 1110 1001. There is a carry out of the MSB, so perform end-around carry: add the carry to the LSB: 10 1110 1001 + 1 = 10 1110 1010. The result is negative (MSB = 1), so take the 1's complement to obtain magnitude: 01 0001 0101_2 = 115_{16}. Thus the 1's complement result is -115_{16}. 2's complement workflow: add the two 10-bit 2's complement values: 11 0110 0101 + 11 1000 0110 = 1 10 1110 1011. Discard the carry out of the MSB. The result is 10 1110 1011_2, which is negative (MSB = 1). To find the magnitude, take the 2's complement: invert bits → 01 0001 0100, add 1 → 01 0001 0101_2 = 115_{16}. Thus the 2's complement result is -115_{16}. Verification in decimal: $-9B_{16}$ = -155_{10}, $-37a_{16}$ = -890_{10}, and -155 - 890 = -1045_{10} = -415_{16} = -115_{16}? Wait, -1045_{10} in hex is -415_{16}, not -115_{16}. The discrepancy indicates an arithmetic error in the binary addition. Re-evaluating: $9B_{16}$ = 155_{10}, $37a_{16}$ = 890_{10}. The sum -155 - 890 = -1045_{10}. 1045_{10} = 0x415 = 0100 0001 0101_2 (11 bits). Therefore the correct 10-bit result cannot represent -1045 because the range of 10-bit 2's complement is -512 to +511. Hence 10 bits are insufficient; we must use at least 12 bits (range -2048 to +2047). Using 12 bits: $9B_{16}$ = 0000 1001 1011_2; -9B in 1's complement = 1111 0110 0100_2, in 2's complement = 1111 0110 0101_2. $37a_{16}$ = 0011 0111 1010_2; -37a in 1's complement = 1100 1000 0101_2, in 2's complement = 1100 1000 0110_2. 1's complement addition: 1111 0110 0100 + 1100 1000 0101 = 1 1011 1110 1001. End-around carry: 1011 1110 1001 + 1 = 1011 1110 1010. Negative result; magnitude = 1's complement = 0100 0001 0101_2 = 415_{16}. So result = -415_{16}. 2's complement addition: 1111 0110 0101 + 1100 1000 0110 = 1 1011 1110 1011. Discard carry: 1011 1110 1011. Negative; magnitude = 2's complement = 0100 0001 0101_2 = 415_{16}. Result = -415_{16}. This matches decimal -1045_{10} = -415_{16}. Therefore the correct answer is -415_{16}. The earlier 10-bit attempt was invalid due to overflow; the complete workflow must select a sufficient bit width (12 bits) before performing complement arithmetic.

Marking points:

• 1 mark: Correctly convert -9B and -37a to binary and choose a sufficient bit width (at least 12 bits) to avoid overflow, with sign extension.

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Correctly form the 1's complement representations of both operands at the chosen width.

• 1 mark: Correctly perform 1's complement addition including end-around carry and interpret the negative result to obtain -415_{16}.

• 1 mark: Correctly form the 2's complement representations and perform 2's complement addition, discarding the final carry, to obtain -415_{16}.

• 1 mark: Show verification (e.g., decimal check -155 - 890 = -1045 = -415_{16}) and state the final result clearly.

Model confidence: high.

Model note: The question text uses 'Brr' and '37a' which are interpreted as hexadecimal numbers 9B and 37A respectively. The subtraction is interpreted as (-9B) - (37A). A 10-bit representation is insufficient because the result -1045 exceeds the 10-bit 2's complement range; 12 bits are required. The answer assumes 12-bit two's complement arithmetic. No figure or table was supplied.

QUESTION 4 (5 marks)

Source: CSC429-2016-06, page [5]

Unofficial model answer:

a) A mapping function is the rule that determines where a block of main memory can be placed in the cache. It specifies how the main-memory block number is translated into a cache line (or set) number, and consequently how the address is partitioned into tag, line/set, and word fields for cache lookup.

b) Direct mapping: each main-memory block maps to exactly one fixed cache line, given by (block number) mod (number of cache lines). The address is divided into tag, line, and word fields; only one tag comparison is needed, and no replacement policy is required because there is only one possible location. It is simple and cheap but suffers from conflict misses when two frequently used blocks map to the same line. Associative mapping: a main-memory block can be placed in any cache line. The address is divided into tag and word fields only (no line field); the tag must be compared against the tags of all lines in parallel, requiring associative memory/comparators. It is flexible and reduces conflict misses but needs a replacement policy and more complex, costly hardware. (Fully associative is the general case; set-associative is a compromise.)

c) Given: address = 28 bits, word size = 4 bits, line size = 10 bits. Assuming the line size of 10 bits means 2^10 = 1024 words per line (a common interpretation when 'line' is given in bits as a field width), the word offset field = 10 bits. Direct mapping: address = tag + line + word. The number of cache lines is not given, so the line field width L cannot be determined; therefore tag size = 28 - L - 10 = (18 - L) bits, where L = log2(number of cache lines). Associative mapping: address = tag + word (no line field), so tag size = 28 - 10 = 18 bits. If instead 'line = 10 bit' is interpreted as the line number field being 10 bits, then direct-mapping tag = 28 - 10 - 10 = 8 bits and associative-mapping tag = 28 - 10 = 18 bits. The question as stated is ambiguous/underspecified for direct mapping; the associative result is 18 bits under the word-offset interpretation.

Marking points:

• 1 mark: (a) correct definition of mapping function as the rule placing main-memory blocks into cache lines and partitioning the address.

• 2 marks: (b) one mark for direct mapping (fixed line = block mod number of lines, tag+line+word, one comparison, conflict misses); one mark for associative mapping (any line, tag+word only, parallel tag comparison, replacement policy, fewer conflict misses).

• 1 mark: (c) correct identification of word offset = 10 bits and associative tag = 28 - 10 = 18 bits.

• 1 mark: (c) correct direct-mapping expression/tag calculation with stated assumption (e.g., tag = 18 - L bits if line field L unknown, or 8 bits if line field = 10 bits), showing awareness that the number of cache lines/line field is needed.

Model confidence: medium.

Model note: The question text gives 'line = 10 bit' without specifying whether this is the line size (offset width) or the line-number field width, and does not give the number of cache lines. The answer states both interpretations and the resulting tag sizes. Word size of 4 bits is taken as the word offset unit; if 'word size' instead meant 4-bit words and line size 10 bits = 1024 words, the offset is 10 bits as used. No figure/table was supplied beyond the text.

© Hak Cipta Universiti Teknologi MARA

QUESTION 5 (10 marks)

Source: CSC429-2017-07, page [5]

Unofficial model answer:

a) Interrupt-driven I/O and programmed I/O differ in how the CPU coordinates with the device. In programmed I/O (polling), the CPU repeatedly reads the device status register in a busy-wait loop and transfers data only when the device is ready; the CPU is fully occupied during the wait and cannot do useful work. In interrupt-driven I/O, the CPU issues an I/O command and continues executing other instructions; when the device completes or needs service, it raises an interrupt, causing the CPU to suspend the current program, save context, and run an interrupt service routine (ISR) to transfer data or handle the event. Thus programmed I/O wastes CPU cycles polling, while interrupt-driven I/O frees the CPU between transfers at the cost of interrupt overhead and hardware complexity.

b) One benefit of programmed I/O is its simplicity and low hardware cost: no interrupt controller, interrupt lines, or context-saving mechanism is required, and the control logic is straightforward to implement and program. This makes it suitable for simple, low-speed devices or small embedded systems where the CPU has no other work to do while waiting.

c) Two disadvantages of multiple interrupts are: (1) Interrupt overhead and reduced throughput — each interrupt requires saving/restoring CPU context and executing an ISR, so many interrupts consume CPU time that could be used for useful work, and frequent interrupts can degrade system performance. (2) Priority and nesting complexity — with multiple interrupt sources, the system must arbitrate priorities; a low-priority interrupt may be delayed or lost (starved) while higher-priority ones are serviced, and nested interrupts complicate ISR design, increase latency, and raise the risk of race conditions or re-entrancy bugs.

Marking points:

- *1 mark: programmed I/O uses CPU polling/busy-wait; CPU repeatedly checks device status*
- *1 mark: interrupt-driven I/O lets CPU continue other work; device signals CPU via interrupt when ready*
- *1 mark: contrast CPU utilization/overhead (polling wastes cycles vs interrupt overhead/context switch)*
- *1 mark: benefit of programmed I/O — simple/low-cost hardware, no interrupt mechanism needed*
- *1 mark: benefit explained in context (suitable for simple/low-speed devices or systems with no other work)*
- *1 mark: disadvantage 1 — interrupt overhead/context save-restore reduces useful CPU time*
- *1 mark: disadvantage 1 explained (many interrupts degrade throughput/performance)*
- *1 mark: disadvantage 2 — priority arbitration/nesting complexity; low-priority interrupts may be delayed or starved*
- *1 mark: disadvantage 2 explained (increased latency, risk of race conditions/re-entrancy issues)*
- *1 mark: overall clarity and correct use of terminology (ISR, polling, context switch, priority)*

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure/table was supplied, so the answer is based solely on the question text. Mark allocation is indicative and assumes a 10-mark total distributed across subparts (a) 3, (b) 2, (c) 5 as reflected in the marking points.

QUESTION 6 (10 marks)

Source: CSC429-2023-07, page [5]

Unofficial model answer:

a) A symmetric multiprocessor (SMP) organization redrawn from Figure 2: replace the distributed main memories (Memory 1, Memory 2) and the NUMA interconnect with a single shared main memory accessed uniformly by all processors. All processors connect to the same shared main memory through a common bus or interconnection network, and each processor typically has its own local cache. Unlike Figure 2, there is no memory that is 'closer' to one processor than another; every processor has equal access latency to the single shared memory. A typical redraw shows: Processor 1 and Processor 2 (each with local cache) connected via a shared system bus/interconnect to one Main Memory block, with all processors symmetric (no master/slave,

© Hak Cipta Universiti Teknologi MARA

no private main memory).

b) Two examples of tightly coupled MIMD parallel processor architectures: (1) Symmetric Multiprocessor (SMP) — multiple identical processors share a single main memory and I/O, with uniform memory access; (2) Non-Uniform Memory Access (NUMA) multiprocessor — multiple processors with distributed shared memory where access time depends on memory location, but all memories form one shared address space. (Other acceptable examples: clustered multiprocessors with shared memory, or multicore processors with shared cache/memory.)

c) Two disadvantages of tightly coupled MIMD parallel processor architectures: (1) Limited scalability — because all processors contend for a single shared memory/bus, adding more processors increases contention and saturates the interconnect, so performance does not scale well beyond a modest number of processors. (2) Memory access bottleneck and coherence overhead — simultaneous access to shared data causes bus/memory conflicts, and cache-coherence protocols add communication and synchronization overhead, increasing latency and complexity. (Also acceptable: higher cost/complexity of shared hardware, and single point of failure in shared memory.)

Marking points:

• *a) 1 mark: single shared main memory shown (no distributed/private main memories)*
• *a) 1 mark: all processors connected symmetrically to the shared memory via a common bus/interconnect*
• *a) 1 mark: each processor shown with its own local cache (or equivalent) and uniform access to memory*
• *b) 1 mark: Symmetric Multiprocessor (SMP) identified as a tightly coupled MIMD example*
• *b) 1 mark: NUMA (or another valid tightly coupled MIMD, e.g., shared-memory cluster/multicore) identified as a second example*
• *c) 1 mark: first disadvantage (e.g., limited scalability due to shared memory/bus contention)*
• *c) 1 mark: explanation/elaboration of first disadvantage*
• *c) 1 mark: second disadvantage (e.g., memory access bottleneck / cache-coherence and synchronization overhead)*
• *c) 1 mark: explanation/elaboration of second disadvantage*
• *Total: 10 marks*

Model confidence: medium.

Model note: Figure 2 is only available as a caption/extracted text; the redraw in (a) is described textually because the actual diagram cannot be reproduced. The answer assumes the standard CSC429 definition of tightly coupled MIMD (shared memory, e.g., SMP and NUMA). Mark allocation is inferred, not official.

QUESTION 7 (10 marks)

Source: CSC429-2025-02, page [5]

Unofficial model answer:

a) SISD diagram (described, since a figure cannot be drawn in JSON): Draw one MU block at the top, one PU block below it, and one CU block to the left of the PU. A single arrow labelled IS runs from MU into CU (the instruction stream), and a single arrow labelled DS runs from MU into PU (the data stream). CU sends a control arrow to PU. Only one instruction stream and one data stream exist, and instructions are executed sequentially — this is the classic von Neumann uniprocessor. (Limitation: the original figure cannot be reproduced here; the description gives the required blocks and labelled streams.)

b) Parallel architecture: A parallel architecture is a computer organization in which multiple processing elements (PUs/cores) cooperate to execute a program, so that several operations are performed simultaneously rather than one at a time. It is characterised by more than one instruction stream and/or more than one data stream (as in SIMD, MISD and MIMD of Flynn's taxonomy), and by hardware for communication/synchronisation between the processing elements (shared memory or an interconnection network). Parallelism may be at instruction level (pipelining, superscalar) or at processor level (multicore,

multiprocessor, cluster). The aim is higher throughput and reduced execution time by exploiting concurrency.

c) One benefit of parallel architecture: Increased performance/throughput — because work is divided among several processing elements, a task can be completed in less time than on a single processor (speedup), and the system can handle larger or more numerous problems. (Other acceptable benefits: better scalability, improved reliability/fault tolerance, better price-performance using many commodity processors.)

d) Loosely-coupled vs strongly-coupled MIMD: In a loosely-coupled (distributed-memory) MIMD system each processor has its own private local memory and its own copy of the operating system; processors communicate by message passing over a network, so memory access to another node is slow and there is no shared global address space. Example: a cluster of workstations or a network of autonomous computers. In a strongly-coupled (shared-memory) MIMD system all processors share a single global main memory and typically one operating system; processors communicate by reading/writing shared variables, so communication is fast but requires synchronisation (locks, semaphores) to avoid race conditions. Example: a symmetric multiprocessor (SMP) multicore server with a common RAM.

Marking points:

• *a) 3 marks: 1 mark for correct MU, PU and CU blocks; 1 mark for a single IS arrow from MU to CU; 1 mark for a single DS arrow from MU to PU (sequential single stream implied).*

• *b) 3 marks: 1 mark for definition of simultaneous execution by multiple processing elements; 1 mark for reference to multiple instruction/data streams (Flynn) or levels of parallelism; 1 mark for communication/synchronisation or the goal of higher throughput.*

• *c) 2 marks: 1 mark for naming a valid benefit (e.g. speedup/throughput); 1 mark for a correct explanation of why it arises.*

• *d) 2 marks: 1 mark for the contrast (private memory + message passing vs shared memory + shared variables/one OS); 1 mark for a correct example of each (cluster/workstation network vs SMP/multicore shared-memory system).*

Model confidence: high.

Model note: The SISD figure is described in words because a diagram cannot be rendered in JSON; the original figure was not supplied, so the layout follows the standard textbook SISD block diagram. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 8 (10 marks)

Source: CSC429-2017-07, page [3]

Unofficial model answer:

a) i) Hexadecimal workflow: Align the two hex operands: AA and CD. Add column by column from the least significant digit: A + D = 10 + 13 = 23 decimal = 17 hex, so write 7 and carry 1. Next column: A + C + carry 1 = 10 + 12 + 1 = 23 decimal = 17 hex, so write 7 and carry 1. The final carry gives the most significant digit 1. Therefore AA + CD = 177_{16}. a) ii) Decimal workflow: Convert each hex operand to decimal: AA_{16} = 10×16 + 10 = 170_{10}; CD_{16} = 12×16 + 13 = 205_{10}. Add the decimal values: 170 + 205 = 375_{10}. Check by converting the hex result: 177_{16} = 1×16² + 7×16 + 7 = 256 + 112 + 7 = 375_{10}. b) i) Sign-and-magnitude workflow: Interpret IA as $1A_{16}$ (the letter I is not a valid hex digit; assume a typographical error for 1). Convert $1A_{16}$ to 8-bit binary: 0001 1010. Convert $3D_{16}$ to 8-bit binary: 0011 1101. For subtraction y = $1A_{16}$ − $3D_{16}$, first find the sign-and-magnitude representation of the subtrahend $3D_{16}$: since $3D_{16}$ is positive, its sign bit is 0 and magnitude is 0011 1101, giving 0011 1101. To subtract using sign-and-magnitude, take the 2's complement of the subtrahend to form its negative: invert bits of 0011 1101 → 1100 0010, add 1 → 1100 0011. This is the 2's complement representation of $-3D_{16}$. Add to the minuend in 8-bit 2's complement: 0001 1010 + 1100 0011 = 1101 1101. The result 1101 1101 has sign bit 1, so it is negative. Its magnitude in 2's complement is obtained by taking 2's complement again: invert 1101 1101 → 0010 0010, add 1 → 0010 0011 = 23_{16} = 35_{10}. Thus y = -23_{16} = -35_{10}. In sign-and-magnitude representation, y is 1 0100011 (sign bit 1, magnitude 0100011_2 = 23_{16}). b) ii) 1's and 2's complement workflow: Using 8-bit representation, $1A_{16}$ = 0001 1010 and $3D_{16}$ = 0011 1101. 1's complement of $3D_{16}$: invert all bits → 1100 0010. 2's complement of $3D_{16}$: add 1 to the 1's complement →

1100 0011. Perform subtraction by adding the 2's complement of the subtrahend: 0001 1010 + 1100 0011 = 1101 1101. Since the sign bit is 1, the result is negative. To express the result in 1's complement, take the 1's complement of the magnitude: the 2's complement result 1101 1101 corresponds to −(0010 0011) in 2's complement; in 1's complement, the negative number is formed by inverting the positive magnitude 0010 0011 → 1101 1100. Thus y in 1's complement is 1101 1100. In 2's complement, y is 1101 1101. Verification: $1A_{16}$ − $3D_{16}$ = 26_{10} − 61_{10} = -35_{10} = -23_{16}. The 8-bit 2's complement of -35_{10} is 1101 1101, and the 1's complement is 1101 1100, which match the computed results.

Marking points:

• *1 mark: Correct hexadecimal addition AA + CD = 177_{16} with carry shown.*

• *1 mark: Correct conversion of AA and CD to decimal (170 and 205).*

• *1 mark: Correct decimal sum 375_{10} and verification with hex result.*

• *1 mark: Correct interpretation of IA as $1A_{16}$ (or clear statement of assumption).*

• *1 mark: Correct 8-bit binary for $1A_{16}$ and $3D_{16}$.*

• *1 mark: Correct sign-and-magnitude representation of the result y = -23_{16} (sign bit 1, magnitude 0100011).*

• *1 mark: Correct 1's complement of $3D_{16}$ (1100 0010) and 2's complement (1100 0011).*

• *1 mark: Correct 2's complement subtraction yielding 1101 1101.*

• *1 mark: Correct 1's complement representation of the result (1101 1100).*

• *1 mark: Correct 2's complement representation of the result (1101 1101) with verification.*

Model confidence: medium.

Model note: The question text contains 'IA' which is not a valid hexadecimal digit; it is assumed to be a typographical error for '1A'. The answer uses 8-bit representations for parts b(i) and b(ii) as is standard for such problems. If a different bit width is intended, the representations would differ. The marking scheme is unofficial and allocated based on the 10-mark total.

QUESTION 9 (10 marks)

Source: CSC429-2017-07, page [4]

Unofficial model answer:

a) Three types of interfaces to external devices: (1) Programmed I/O (polling) interface — the CPU repeatedly checks the status register of the I/O module and transfers data through the CPU; simple but wastes CPU cycles. (2) Interrupt-driven I/O interface — the I/O module raises an interrupt request when a device is ready or has completed a transfer, so the CPU need not poll; improves CPU utilisation but each transfer still passes through the CPU. (3) Direct Memory Access (DMA) interface — a DMA controller takes over the bus and transfers blocks of data directly between the device and main memory without CPU involvement per word, generating an interrupt only when the whole block is complete; suited to high-speed devices. (Other acceptable interfaces: memory-mapped I/O vs isolated I/O, or serial vs parallel interfaces.)

b) Classification of devices to their I/O module: (i) Network interface card — a high-speed device, typically handled by a DMA-capable I/O module (or interrupt-driven module) because it moves large blocks of data. (ii) Scanner — a slow character/block device, usually connected through an interrupt-driven or programmed I/O module (e.g., USB/SCSI controller). (iii) Touch Screen — a slow input device, served by an interrupt-driven I/O module (each touch generates an interrupt). (iv) Register — not an external I/O device; it is internal CPU storage, so it does not belong to any external I/O module (it is accessed directly by the CPU, not via an I/O interface).

Marking points:

• *a) 1 mark: correctly names and briefly describes Programmed I/O (polling) interface.*

• *a) 1 mark: correctly names and briefly describes Interrupt-driven I/O interface.*

• *a) 1 mark: correctly names and briefly describes DMA interface.*

• *a) 1 mark: compares/contrasts the three interfaces (e.g., CPU involvement, speed, suitability).*

• *a) 1 mark: uses correct terminology (status register, interrupt request, DMA controller, bus).*

© Hak Cipta Universiti Teknologi MARA

• b) 1 mark: Network interface card classified to DMA/high-speed I/O module with justification.

• b) 1 mark: Scanner classified to interrupt-driven/programmed I/O module with justification.

• b) 1 mark: Touch Screen classified to interrupt-driven I/O module with justification.

• b) 1 mark: Register identified as internal CPU storage, not an external I/O device.

• b) 1 mark: all four classifications consistent and clearly stated.

Model confidence: medium.

Model note: The question text for part b) is partially garbled (e.g., '1/0' for I/O, missing item numbering). The classification assumes standard I/O module types (programmed, interrupt-driven, DMA). 'Register' is interpreted as a CPU register; if the paper intended an I/O register/port, the answer would differ. No figure or table was supplied.

QUESTION 10 (10 marks)

Source: CSC429-2024-01, page [3]

Unofficial model answer:

a) The two processes in the machine cycle governed by main memory are the Fetch phase and the Memory (data transfer) phase. In the Fetch phase, main memory supplies the instruction word addressed by the Program Counter (PC) to the CPU; the memory is the source and the instruction is placed on the data bus into the Instruction Register (IR). In the Memory phase (also called the memory read/write or data transfer phase), main memory acts as the source or destination for operands: a Load/Read transfers a data word from memory to a CPU register, while a Store/Write transfers a register value to memory. Both processes are 'governed' by main memory because the memory address register (MAR) selects the location and the memory control signals (Read/Write) determine the direction of transfer; the CPU initiates but the timing and completion depend on memory access.

b) Two operations comprised in the Execute phase are: (i) arithmetic/logic operations performed by the ALU (e.g., ADD, SUB, AND, OR, compare) on operands taken from registers or memory; and (ii) data transfer operations such as loading a register from memory or storing a register to memory (and register-to-register moves). Other acceptable examples include incrementing the PC, setting condition flags, or a branch/jump that updates the PC. The Execute phase decodes and carries out the operation specified by the instruction fetched in the previous phase.

c) During the Fetch cycle, the Program Counter (PC) holds the address of the next instruction to be fetched. Its content is copied to the Memory Address Register (MAR) so memory can be addressed; after the instruction is fetched, the PC is incremented (PC <- PC + 1, or by the instruction length) so it points to the following instruction. The Instruction Register (IR) receives and holds the fetched instruction word from main memory via the Memory Data Register (MDR)/data bus. The IR then supplies the instruction to the control unit and decoder, which interprets the opcode and generates the control signals for the Execute phase. Thus the PC is the address pointer that advances, while the IR is the holding register for the current instruction.

Marking points:

• a) 1 mark: identifies Fetch phase as memory-governed (instruction read from memory using PC/MAR).

• a) 1 mark: identifies Memory/data-transfer phase as memory-governed (operand read/write between CPU and memory).

• a) 1 mark: explains role of MAR/address and Read/Write control in selecting memory location and transfer direction.

• b) 1 mark: names one valid Execute-phase operation (e.g., ALU arithmetic/logic operation).

• b) 1 mark: names a second valid Execute-phase operation (e.g., load/store data transfer or branch/PC update).

• b) 1 mark: briefly explains that Execute carries out the decoded operation.

• c) 1 mark: states PC holds address of next instruction and is copied to MAR.

• c) 1 mark: states PC is incremented after fetch to point to next instruction.

• c) 1 mark: states IR receives/holds the fetched instruction from memory.

• c) 1 mark: states IR feeds the control unit/decoder to interpret opcode and generate control signals.

© Hak Cipta Universiti Teknologi MARA

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure/table was supplied beyond the question text, so the answer is based on standard von Neumann machine-cycle descriptions. Mark allocation is indicative; the actual scheme may distribute the 10 marks differently.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2023-07, page [6]

Unofficial model answer:

Figure 3 represents the expression 242 − (B + AC), i.e. 242 minus the sum of register B and the product of registers A and C. Registers A, B and C are read-only; x, y and z are temporaries; the final result must end in register y. The three programs below use only the mnemonics listed in Table 1.

Zero-address program: With a stack machine, operands are pushed and the arithmetic instructions pop their operands and push the result. The constant 242 is pushed first, then B, then A and C are pushed and MUL is executed to leave AC on the stack; ADD then combines B and AC; SUB computes 242 − (B + AC); finally pop stores the top of stack into y. Program: PUSH 242; PUSH B; PUSH A; PUSH C; MUL; ADD; SUB; POP y. Comments: each PUSH places an operand on the stack; MUL replaces A and C by A*C; ADD replaces B and A*C by B+AC; SUB replaces 242 and (B+AC) by 242−(B+AC); POP y writes the result to y.

One-address program: A one-address machine uses an implicit accumulator. LOAD copies a memory/register operand into the accumulator; ADD, SUB, MUL and DIV combine the accumulator with the named operand and leave the result in the accumulator; STOR copies the accumulator to the named destination. Because A, B and C cannot be modified, the product AC is formed in a temporary: LOAD A; MUL C; STOR x (x = A*C); LOAD B; ADD x (accumulator = B + AC); STOR z (z = B + AC); LOAD 242; SUB z (accumulator = 242 − (B + AC)); STOR y. Comments: the first three instructions compute AC into x; the next two compute B + AC into z; the last three compute the final subtraction and store the answer in y.

Three-address program: Each three-address instruction has the form OP dest, src1, src2 and does not destroy its sources. Using temporaries x, y and z: MUL x, A, C (x = A*C); ADD z, B, x (z = B + AC); SUB y, 242, z (y = 242 − (B + AC)). Comments: MUL forms the product without changing A or C; ADD forms the parenthesised sum without changing B; SUB performs the final subtraction and places the required final answer directly in y. No STOR is needed because the destination of the last instruction is y.

Marking points:

• 2 marks: correctly identifies the expression as 242 − (B + AC) and respects the read-only constraint on A, B, C (uses only x, y, z as temporaries).

• 6 marks: zero-address program — correct use of PUSH/POP and stack order (2), correct MUL for AC (1), correct ADD for B+AC (1), correct SUB order 242 − (B+AC) (1), final POP y (1).

• 6 marks: one-address program — correct LOAD/STOR/accumulator usage (2), computes AC into a temporary without modifying A or C (1), computes B+AC (1), computes 242 − (B+AC) in the right order (1), final STOR y (1).

• 4 marks: three-address program — MUL x, A, C (1), ADD z, B, x (1), SUB y, 242, z (1), final answer in y and no source registers modified (1).

• 2 marks: correct assembly notation and meaningful comments throughout (award proportionally across the three programs).

Model confidence: high.

Model note: Figure 3 is supplied only as extracted text '242 - (B + AC)'; the answer assumes this is the intended expression. Table 1 lists only mnemonics, not operand syntax, so standard zero-, one- and three-address operand forms are assumed. The one-address column in the extracted table omits MUL; the answer assumes MUL is available on the one-address machine because the question requires computing AC. This is an unofficial model answer, not an official UTM marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA